

LeetCode 1292: Maximum Side Length of a Square with Sum Less than or Equal to Threshold

Amazon

IMC

Binary Search

Prefix Sum

1. Problem Analysis & Example

The problem asks us to find the maximum side length L of a square within a matrix such that the sum of the elements in the square is less than or equal to a given threshold k .

Example: Given $k = 8$ and the matrix:

1	1	3
2	4	3
2	2	2

A square of size 3×3 is not possible because the total sum exceeds 8. However, a square of size 2×2 (for example, the bottom-left elements) exists with a sum ≤ 8 . Hence, the output is 2.

2. Approach & Intuition

Brute Force: The naive approach involves trying every possible starting position for the top-left corner of the square, and for every square, computing the sum. The time complexity for this is $O(n \times m \times L^2)$, where $L = \min(n, m)$. This is too slow for large inputs.

Key Observation: Monotonicity

Can a square of size 3 exist? If YES, then a square of size 2 and size 1 must also exist. Conversely, if a square of size 3 CANNOT exist, then squares of sizes 4, 5, and 6 cannot exist either.

The pattern of valid square lengths looks like: **YES, YES, YES, NO, NO, NO**.

This monotonic property allows us to apply **Binary Search** on the length of the square.

We create a helper function `check(len)` to see if a square of side `len` exists satisfying the property. We iterate through the matrix using row offsets (`roff`) and column offsets (`coff`), mapping the coordinates to:

- $r_1 = roff, r_2 = roff + len - 1$
- $c_1 = coff, c_2 = coff + len - 1$

To compute the sum in $O(1)$ time, we utilize a 2D Prefix Sum array.

3. C++ Implementation

Below is the fully functional and optimized C++ solution implementing the Binary Search and Prefix Sum logic.

```

class Solution {
public:
    bool check(int len, int k, int n, int m, vector<vector<int>>& pref){
        if(len==0) return true;

        for(int roff=0;roff+len<=n;roff++){
            for(int coff=0;coff+len<=m;coff++){
                int r1=roff;
                int c1=coff;
                int r2=roff+len-1;
                int c2=coff+len-1;

                int sum=pref[r2][c2];
                if(r1>0){
                    sum-=pref[r1-1][c2];
                }
                if(c1>0){
                    sum-=pref[r2][c1-1];
                }

                if(r1>0 && c1>0){
                    sum+=pref[r1-1][c1-1];
                }

                if(sum<=k) return true;
            }
        }
        return false;
    }

    int maxSideLength(vector<vector<int>>& mat, int k) {
        int n=mat.size();
        int m=mat[0].size();
        int ans=0;

        vector<vector<int>> pref(n,vector<int>(m));

        for(int i=0;i<n;i++){
            for(int j=0;j<m;j++){
                pref[i][j]=mat[i][j];
                if(i>0){
                    pref[i][j]+=pref[i-1][j];
                }
            }
        }
    }

```

```

        if(j>0){
            pref[i][j]+=pref[i][j-1];
        }

        if(i>0 && j>0){
            pref[i][j]-=pref[i-1][j-1];
        }
    }
}

int l=0;
int h=min(n,m);
while(l<=h){
    int mid=l+(h-l)/2;
    if(check(mid,k,n,m,pref)){
        ans=mid;
        l=mid+1;
    }
    else{
        h=mid-1;
    }
}

return ans;
}
};

```